



OOP and Data Structure



Easy English Notes – Unit 1 to Unit 5



Unit 1 – Concept of OOPS



Unit 2 – Data Encapsulation and Inheritance



Unit 3 – Polymorphism



Unit 4 – Data Structure (Stack)



Unit 5 – Data Structure (Queue)



UNIT 1 - Concept of OOPS



1.1 Procedure Oriented Programming (POP) vs Object Oriented Programming (OOP)

Point	POP	OOP
Divided into	Small parts called functions	Parts called objects
Importance	Given to functions and the sequence of actions	Given to data , because it works like the real world
Approach	Top Down	Bottom Up
Access specifiers	None	Public, Private, Protected etc.
Data moving	Data moves freely from function to function	Objects talk to each other through member functions
Expansion	Adding new data and functions is not easy	Adding new data and functions is easy
Data access	Most functions use global data , freely accessible	Data cannot move easily; it can be public or private, so access is controlled
Data hiding	No proper way, so less secure	Data hiding is available, so more secure
Overloading	Not possible	Possible: function overloading and operator overloading
Examples	C, VB, FORTRAN, Pascal	C++, Java, VB.NET, C#.NET



POP is like a factory assembly line (step by step). OOP is like a team where every person (object) has own data and own job.



1.2 Header Files in C++

- Header files give **library functions, data types, macros** etc. to your program.
- They are added with the preprocessor directive **#include**, which tells the compiler to process these files **before compilation**.
- In C++, header files may or may not end with **.h**.

Header file	Description
<code><iostream></code>	Input and output stream using <code>cin</code> and <code>cout</code>
<code><fstream></code>	Read data from a file and write data into a file
<code><new></code>	Dynamic memory allocation methods
<code><vector></code>	Definition of the vector class container
<code><map></code>	Definition of the map class container; fast access to a value using its key

1.3 Data Types in C++

◆ Primitive (built-in) Data Types

- Built-in or predefined types that can be used directly to declare variables, like `int`, `char`, `float`.

Type	Keyword	Meaning	Memory
Integer	<code>int</code>	Whole numbers	Notes say 2 bytes, range -2147483648 to 2147483647 (that range is for 4 bytes)
Boolean	<code>bool</code>	Stores logical value true or false	1 byte
Floating point	<code>float</code>	Single-precision decimal values	4 bytes
Double floating point	<code>double</code>	Double-precision decimal values	8 bytes
void	<code>void</code>	Means without any value ; used for functions that return no value	-
Character	<code>char</code>	Stores a letter	1 byte

💡 In old Turbo C++ `int` is 2 bytes. In modern compilers `int` is 4 bytes, which matches the range given above.

📁 Example: Data Types (Turbo C++)

```
#include<iostream.h>
#include<conio.h>

void main()
{
    int a = 10;
    float b = 5.5;
    double c = 12.345678;
    char d = 'A';
    clrscr();
    cout << "Integer    : " << a << endl;
    cout << "Float      : " << b << endl;
    cout << "Double     : " << c << endl;
    cout << "Character  : " << d << endl;
    cout << "Size of int   = " << sizeof(int) << " bytes" << endl;
    cout << "Size of char  = " << sizeof(char) << " byte" << endl;
    cout << "Size of float = " << sizeof(float) << " bytes" << endl;
    cout << "Size of double = " << sizeof(double) << " bytes" << endl;
    getch();
}
```

Output:

```
Integer    : 10
Float      : 5.5
Double     : 12.3457
Character  : A
Size of int   = 2 bytes
Size of char  = 1 byte
Size of float = 4 bytes
Size of double = 8 bytes
```

abc 1.4 Concept of String

◆ 1.4.1 Character Array

- `char` stores letters. A character array stores a **collection of characters (a string)**.
- A `char` takes **1 byte**, so memory = 1 x number of elements.
- Each character has an **index**; the first character is index **0**, then 1, 2, 3...
- The **null character** `\0` marks the **end of the string**. It is stored after the last character.
- Example: the string "character" is stored like this:

```
index : 0  1  2  3  4  5  6  7  8  9
value : c  h  a  r  a  c  t  e  r  \0
                                     ↑ end of string
```

◆ 1.4.2 Pointer to Character Array

- When a character pointer or array is given to `cout`, it prints the **complete string** (till the null character), not the address.

```
#include<iostream.h>
#include<conio.h>
void main()
{
    clrscr();
    char ch[] = "abc";    // character array
    cout << ch << endl;  // prints abc
    getch();
}
```

Output:

```
abc
```

🧩 1.5 Concepts of Class and Objects

◆ Objects

- Objects are the **basic run-time entities** of an OOP system. An object can be a person, a place, a bank account, a table of data or anything the program handles.
- A problem is solved in terms of objects and how they **communicate**.
- Objects take **space in memory** and have an **address**.
- Objects interact by **sending messages**. Example: a "customer" object asks the "account" object for the bank balance.
- Each object has **data** and **code to work on the data**.
- Objects need not know each other's data or code; they only need to know the **type of message** and the **type of response**.

◆ Classes

- A **class** is a user-defined data type that holds the data and code of an object.
- **Objects are variables of the type class**. One class can make **any number of objects**.
- A class is a collection of objects of similar type. Example: Mango, Apple, Orange are members of class Fruit.
- `Fruit Mango;` creates an object `Mango` of class `Fruit`.

📱 Mobile is a class; iPhone, Redmi, Samsung are objects of that class.

VS Class vs Object

Class	Object
Blueprint for declaring and creating objects	Instance of a class; uses the variables and methods of the class
No memory is allocated; no physical existence	Memory is allocated (in heap memory) when created
Declared only once	Many objects can be created from one class
Logical entity	Physical entity
Cannot be manipulated (not in memory)	Can be manipulated
Example: Mobile	Example: iPhone, Redmi, Samsung

Example: Class and Object (Turbo C++)

```
#include<iostream.h>
#include<conio.h>

class mobile
{
    char brand[20];
    int price;
public:
    void set_data()
    {
        cout << "Enter brand and price: ";
        cin >> brand >> price;
    }
    void show_data()
    {
        cout << "Brand : " << brand << endl;
        cout << "Price : " << price << endl;
    }
};

void main()
{
    mobile m1, m2;           // m1 and m2 are objects of class mobile
    clrscr();
    m1.set_data();
    m2.set_data();
    m1.show_data();
    m2.show_data();
    getch();
}
```

Output:

```
Enter brand and price: iPhone 80000
Enter brand and price: Redmi 15000
Brand : iPhone
Price : 80000
Brand : Redmi
Price : 15000
```

★ Features of OOP

- The main reason for OOP is to **remove the flaws** of the procedural approach.

- OOP treats data as a **critical element** and does not let it flow freely around the system.
- It ties data closely to the functions that work on it and **protects data from accidental change** by outside functions.
- Features:
 - Emphasis is on **data**, not procedure.
 - Programs are divided into **objects**.
 - Data structures are designed to describe the objects.
 - Functions that work on the data of an object are tied together with it.
 - Data is **hidden** and cannot be accessed by external functions.
 - Objects talk to each other through functions.
 - New data and functions can be **added easily**.
 - Follows the **bottom-up** approach.

 OOP is an approach that gives a way of modularizing programs by creating partitioned memory areas for both data and functions, which can be used as templates for making copies of such modules on demand.

Basic Concepts of OOP

1. Objects 2. Classes 3. Data abstraction and encapsulation 4. Inheritance 5. Polymorphism 6. Dynamic binding 7. Message passing

◆ Encapsulation

- **Wrapping up data and functions into a single unit (class)** is called encapsulation.
- Data is not accessible to the outside world; only the functions inside the class can access it. These functions are the **interface** between the object's data and the program.
- Protecting data from direct access is called **data hiding** or **information hiding**.

 A medicine capsule keeps the powder inside one cover. Same idea: data and functions inside one class.

◆ Abstraction

- Showing only the **essential features** without the background details.
- A class is a list of **data members** and the functions (**methods / member functions**) that work on them.

 You only press the accelerator to speed up; you do not need to know how the engine works.

◆ Inheritance

- The process in which objects of one class **acquire the properties** of objects of another class.
- It supports **hierarchical classification**. Example: bird, flying bird, robin.
- It gives **reusability**: add new features to an existing class without changing it, by making a **new derived class**. The new class has the features of both.

◆ Polymorphism

- Greek word: "**more than one form**". One operation can behave differently in different cases.
- Example:  adds two numbers, but joins (concatenates) two strings. Making an operator behave differently = **operator overloading**.
- Using one function name for different tasks (different number or type of arguments) = **function overloading**.

- It lets objects with different internal structure share the **same external interface**. It is used a lot in inheritance.

◆ Dynamic Binding

- **Binding** = linking a function call to the code that will run.
- **Dynamic (late) binding** = the code for a call is decided **at run time**.
- It is linked with polymorphism and inheritance.

◆ Message Passing

- An OOP program is a set of objects that **communicate** with each other, like people passing messages.
- Basic steps: **1. Create classes** (objects and their behaviour), **2. Create objects** from the classes, **3. Set up communication** between objects.



Benefits of OOP

- **Inheritance** removes repeated code and reuses existing classes.
- Programs are built from **standard working modules**, so development time is saved and productivity is higher.
- **Data hiding** helps to build secure programs.
- Many **instances of an object** can exist together without interfering.
- Objects in the problem can be **mapped** to objects in the program.
- Work in a project is easy to **divide** by objects.
- Data-centred design captures more detail of a model.
- Systems can be **upgraded** easily from small to large.
- Message passing makes interface descriptions with external systems simple.
- **Software complexity** is easy to manage.



Specifying a Class

- A class combines **related data and functions**. It makes a data type used to create objects.
- A class has two parts: **1. Class declaration, 2. Class function definition**.

◆ Class Declaration

```
class name_of_class
{
private:
    variable declaration;    // data member
    function declaration;    // member function (method)
protected:
    variable declaration;
    function declaration;
public:
    variable declaration;
    function declaration;
};
```

- `class` keyword says we are making a new data type; the class name follows it.
- `private`, `protected` and `public` are **access specifiers**.
- **Private** members can be used only **inside the class** (this gives data hiding). **Public** members can be used **outside the class** too.
- Private data is safe from accidental change.
- Functions are usually public so they can be called from outside, but this is not a rule.

◆ Class Function Definition

- Member functions can be written in two ways: **(a) inside the class** or **(b) outside the class using the scope resolution operator ::**.
- **Inside the class:** no class name is needed with the function name. Such small functions are called **inline functions**; the compiler puts the function body at the place of the call, so execution is faster.
- **Outside the class:** the function's full name is written as `class_name::function_name`.

```
return_type name_of_the_class::function_name(argument list) { body of function }
```

- `::` helps to define a member function outside the class. It is also used to access a **global variable** when a local variable has the same name.

📁 Example of a Class

```
#include<iostream.h>
#include<conio.h>

class student
{
private:
    int reg_no;
    char name[30];
    int age;
public:
    void init_data()           // defined inside the class (inline function)
    {
        cout << "Enter reg no, name and age: ";
        cin >> reg_no >> name >> age;
    }
    void display_data();       // only declared here
};

void student::display_data()  // defined outside the class using ::
{
    cout << "\nReg No : " << reg_no;
    cout << "\nName   : " << name;
    cout << "\nAge    : " << age;
}

void main()
{
    student ob;               // object created
    clrscr();
    ob.init_data();           // access member function
    ob.display_data();        // access member function
    getch();
}
```

Output:

```
Enter reg no, name and age: 101 Raj 19

Reg No : 101
Name   : Raj
Age    : 19
```

! Important Questions

- 1. Explain the difference between POP and OOP.
- 2. Explain Data Types in C++.

- 3. What is Class and Object? Explain the difference between them.
- 4. Explain the concept of Object Oriented Programming.
- 5. Advantages of Object Oriented Programming.

★ Unit 1 Quick Revision

Topic	Remember
POP vs OOP	POP = functions, top down, no hiding. OOP = objects, bottom up, data hiding
Header files	iostream, fstream, new, vector, map; added with <code>#include</code>
Data types	int, bool, float (4 bytes), double (8 bytes), void, char
String	char array ends with <code>\0</code> ; index starts at 0
Class	Blueprint, no memory, logical
Object	Instance of class, memory in heap, physical
7 basic concepts	Objects, Classes, Abstraction and Encapsulation, Inheritance, Polymorphism, Dynamic binding, Message passing
Encapsulation	Wrapping data and functions into one unit
Abstraction	Showing only essential details
Access specifiers	private, protected, public
Member function	Inside class = inline; outside class = use <code>::</code>



UNIT 2 - Data Encapsulation and Inheritance



2.1 Access Control Concepts

- Access modifiers are used for **data hiding**.
- C++ has **three access specifiers**:

Specifier	Who can use the member	Typical use
public	Anywhere , inside and outside the class	Interface of the class
private	Only inside the class	Implementation, hidden from outside
protected	Inside the class and its derived classes	Implementation to be shared with child classes



Public = front gate (everyone). Protected = family only. Private = your own locker.



2.2 Declaring a Simple Class, Member Variables and Member Functions

- A class combines **related data and functions** and makes a data type for creating objects.
- A class has two parts: **Class Declaration** and **Class Function Definition**.

```
class name_of_class
{
private:
    variable declaration;    // data member
    function declaration;    // member function (method)
protected:
    variable declaration;
    function declaration;
public:
    variable declaration;
    function declaration;
};
```

- Private data and functions work **only inside the class**; public ones work **outside also**.
- Functions are usually public, but this is not a rule.

◆ Member Function Definition

- **Inside the class:** no class name needed; these are **inline functions** (compiler puts the body at the call, so it runs faster).
- **Outside the class:** use the **scope resolution operator ::**

```
return_type name_of_the_class::function_name(argument list) { body of function }
```

- **::** also identifies a **global variable** when a local variable has the same name.

```
#include<iostream.h>
#include<conio.h>

class box
{
private:
    int length, breadth;    // member variables (private)
public:
    void set_data(int l, int b)    // defined inside the class (inline)
```

```

{
    length = l;
    breadth = b;
}
int area();                // only declared here
};

int box::area()             // defined outside the class using ::
{
    return length * breadth;
}

void main()
{
    box b1;
    clrscr();
    b1.set_data(10, 5);
    cout << "Area = " << b1.area();
    getch();
}

```

Output:

```
Area = 50
```

12 2.3 Concepts and Use of enum

- An **enumeration** is a user-defined data type. It gives **names to integer constants**, so a program is easy to read and maintain.
- Keyword: **enum**. Syntax: `enum enum_name { const1, const2, ... };`
- Values start from **0** (like an array index). Other values cannot be assigned to an enum type; the compiler gives an error.

```

#include<iostream.h>
#include<conio.h>
enum year { Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec };

void main()
{
    clrscr();
    int i;
    for (i = Jan; i <= Dec; i++)
        cout << i << " ";
    getch();
}

```

Output:

```
0 1 2 3 4 5 6 7 8 9 10 11
```

- Output: 0 1 2 3 4 5 6 7 8 9 10 11

2.4 Data Hiding, Abstraction and Encapsulation

◆ Encapsulation

- **Wrapping data and functions into one unit (class).**
- Data is not accessible to the outside world; only functions of the class can use it. These functions are the **interface** between the data and the program.
- Wrapping data away from direct access = **data hiding / information hiding**.

Features of Encapsulation

- We cannot call a class function directly; we need an **object**.
- The function must use the **member variables** of the class; only then it is encapsulation.
- It **increases security** of data.
- It helps to **control the modification** of data members.

```
#include<iostream.h>
#include<conio.h>
class Encapsulation
{
private:
    int x;           // data hidden from outside world
public:
    void set(int a)   // sets value of x
    {
        x = a;
    }
    int get()         // returns value of x
    {
        return x;
    }
};

void main()
{
    clrscr();
    Encapsulation obj;
    obj.set(5);
    cout << obj.get();    // prints 5
    getch();
}
```

Output:

5

◆ Abstraction

- Showing only the **essential features** and hiding the background details.
- A class is a list of **data members** and the functions (methods) that work on them.

 A driver knows that the accelerator increases speed and the brake stops the car, but does not know how the engine does it. That is abstraction.

Advantages of Data Abstraction

- Avoids writing low-level code.
- Avoids code duplication and increases **reusability**.
- Internal implementation can change **without affecting the user**.
- Increases **security**: only important details are shown.
- Reduces complexity and repeated code, so code is more readable.

```
#include<iostream.h>
#include<conio.h>
class bank
{
    float amt, rate, year;
    float si;           // simple interest
    float NetAmt;
public:
```

```

void read()
{
    cout << " Enter Principle Amount :: ";
    cin >> amt;
    cout << "\n Enter Rate of Interest :: ";
    cin >> rate;
    cout << "\n Enter Number of years :: ";
    cin >> year;
    si = (amt * rate * year) / 100;
    NetAmt = si + amt;
}
void show()
{
    cout << "\n\n Interest : " << si;
    cout << "\n\n Total Amount : " << NetAmt << "\n";
}
};

void main()
{
    clrscr();
    bank b;
    b.read();
    b.show();
    getch();
}

```

Output (sample run):

```

Enter Principle Amount :: 1000
Enter Rate of Interest :: 5
Enter Number of years :: 2

Interest : 100

Total Amount : 1100

```

◆ Data Hiding

- **Hiding the internal data (data members) of an object** so it cannot be accessed directly from outside the class.
- It guarantees **data integrity** and reduces complexity by lowering dependency between parts of software.
- Data hiding is done using the **private** access specifier.

```

#include<iostream.h>
#include<conio.h>
class ABC
{
    int num;                // private by default
public:
    void read();
    void print();
};

void ABC::read()
{
    cout << "Enter any Integer value" << endl;
    cin >> num;
}

void ABC::print()
{

```

```

    cout << "The value is " << num << endl;
}

void main()
{
    clrscr();
    ABC obj;
    obj.read();
    obj.print();
    getch();
}

```

Output (sample run):

```

Enter any Integer value
25
The value is 25

```

VS Data Hiding vs Encapsulation

Data Hiding	Encapsulation
Related to data security	Wrapping up data into a single module
Hides the complexity of the application	Hides complicated and confidential information about the application
Focuses on restricting data usage	Focuses on hiding the complexity of the system
A process and a technique	A sub-process inside the bigger process of data hiding
Data is always private and inaccessible	Encapsulated data can be private or public, as needed

👉 Friend Function

- A **friend function** is defined **outside the class** but can access the **private and protected** members of the class.
- It is **not a member function** of the class.
- Used when two classes need to share their members: make the function a friend of both classes.
- Normal non-member functions cannot access private members; a friend function can.

Characteristics of a Friend Function

- It is **not in the scope** of the class.
- It **cannot be called using an object** of the class; it is called like a normal function.
- It **cannot use member names directly**; it uses an object name and the dot operator.
- It can be declared in the **public or private** part without changing its meaning.
- Usually it has **objects as arguments**.

```

#include<iostream.h>
#include<conio.h>
class abc
{
    int x, y;
public:
    void setdata(int no1, int no2)
    {
        x = no1;
        y = no2;
    }
    friend void big(abc);          // friend function

```

```
};

void big(abc a1)
{
    if (a1.x > a1.y)
        cout << "Big No is : " << a1.x << "\n";
    else
        cout << "Big No is : " << a1.y << "\n";
}

void main()
{
    clrscr();
    abc a;
    a.setdata(10, 20);
    big(a);           // output: Big No is : 20
    getch();
}
```

Output:

```
Big No is : 20
```



2.5 Concepts of Inheritance and Types of Inheritance

- **Inheritance** lets us define a class in terms of another class. It gives **code reuse** and faster development.
- The existing class is the **base class** (parent); the new class is the **derived class** (child).
- It implements an **"is-a" relationship**. Example: dog is-a mammal, mammal is-a animal.
- A class can be derived from **more than one base class**.
- Syntax: `class derived_class : access_specifier base_class`
- If the access specifier is not written, it is **private by default**.

◆ Access Control and Inheritance

- A derived class can access **all non-private members** of its base class.
- Members that derived classes must not use should be **private** in the base class.

◆ Rules for Inheritance

- 1. **Private members** of the base class are never accessible in the child class.
- 2. Child inherits parent **as protected**: parent's protected and public become **protected**.
- 3. Child inherits parent **as private**: parent's protected and public become **private**.
- 4. Child inherits parent **as public**: parent's protected stays **protected** and public stays **public**.

Base class member	Public inheritance	Protected inheritance	Private inheritance
public	public	protected	private
protected	protected	protected	private
private	Not accessible	Not accessible	Not accessible

- Public inheritance is used most; protected and private inheritance are hardly used.

◆ Who Can Access What

Access	Public	Protected	Private
Same class	Yes	Yes	Yes

Access	Public	Protected	Private
Derived classes	Yes	Yes	No
Outside classes	Yes	No	No

◆ What is NOT inherited

- **Constructors, destructors and copy constructors** of the base class.
- **Overloaded operators** of the base class.
- **Friend functions** of the base class.

Types of Inheritance

1. Single 2. Multiple 3. Hierarchical 4. Multilevel 5. Hybrid

◆ 1. Single Inheritance

- **One derived class** inherits from **only one base class**. The simplest form.

```
class A                // base class or parent class
{
    // class body
};
class B : public A // derived class or child class
{
    // class body
};
```

```
A
↓
B
```

Family Example: Father and Son (Single)

```
#include<iostream.h>
#include<conio.h>

class Father                // base class
{
public:
    void show_father()
    {
        cout << "I am Father" << endl;
    }
};

class Son : public Father    // derived class
{
public:
    void show_son()
    {
        cout << "I am Son" << endl;
    }
};

void main()
{
    Son s;
    clrscr();
    s.show_father();        // function of base class
    s.show_son();           // own function
}
```

```
    getch();  
}
```

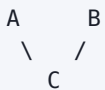
Output:

```
I am Father  
I am Son
```

◆ 2. Multiple Inheritance

- **One derived class** inherits from **two or more base classes**.

```
class A { };           // 1st base class  
class B { };           // 2nd base class  
class C : public A, public B // derived from A and B  
{  
    // class body  
};
```



👨👩👦 Family Example: Mother, Father and Son (Multiple)

```
#include<iostream.h>  
#include<conio.h>  
  
class Mother           // 1st base class  
{  
public:  
    void show_mother()  
    {  
        cout << "I am Mother" << endl;  
    }  
};  
  
class Father           // 2nd base class  
{  
public:  
    void show_father()  
    {  
        cout << "I am Father" << endl;  
    }  
};  
  
class Son : public Mother, public Father // derived from Mother and Father  
{  
public:  
    void show_son()  
    {  
        cout << "I am Son" << endl;  
    }  
};  
  
void main()  
{  
    Son s;  
    clrscr();  
    s.show_mother();  
    s.show_father();  
    s.show_son();  
}
```

```
    getch();  
}
```

Output:

```
I am Mother  
I am Father  
I am Son
```

◆ 3. Hierarchical Inheritance

- **Many derived classes** inherit from **one base class**.

```
class A { };           // base class  
class B : public A { }; // derived from A  
class C : public A { }; // derived from A  
class D : public A { }; // derived from A
```

```
    A  
   / | \  
  B  C  D
```

👨👩👦 Family Example: Father, Son and Daughter (Hierarchical)

```
#include<iostream.h>  
#include<conio.h>  
  
class Father           // base class  
{  
public:  
    void show_father()  
    {  
        cout << "I am Father" << endl;  
    }  
};  
  
class Son : public Father // derived from Father  
{  
public:  
    void show_son()  
    {  
        cout << "I am Son" << endl;  
    }  
};  
  
class Daughter : public Father // derived from Father  
{  
public:  
    void show_daughter()  
    {  
        cout << "I am Daughter" << endl;  
    }  
};  
  
void main()  
{  
    Son s;  
    Daughter d;  
    clrscr();  
    s.show_father();  
    s.show_son();  
    d.show_father();  
    d.show_daughter();  
}
```

```
    getch();  
}
```

Output:

```
I am Father  
I am Son  
I am Father  
I am Daughter
```

◆ 4. Multilevel Inheritance

- A derived class inherits from a class which itself inherits from another class. The super class of one is the sub class of the other.

```
class A { };           // 1st base class  
class B : public A { }; // derived from A  
class C : public B { }; // derived from B
```

```
A  
↓  
B  
↓  
C
```

👴👤👦 Family Example: Grandfather, Father and Son (Multilevel)

```
#include<iostream.h>  
#include<conio.h>  
  
class Grandfather           // 1st base class  
{  
public:  
    void show_grandfather()  
    {  
        cout << "I am Grandfather" << endl;  
    }  
};  
  
class Father : public Grandfather // derived from Grandfather  
{  
public:  
    void show_father()  
    {  
        cout << "I am Father" << endl;  
    }  
};  
  
class Son : public Father      // derived from Father  
{  
public:  
    void show_son()  
    {  
        cout << "I am Son" << endl;  
    }  
};  
  
void main()  
{  
    Son s;  
    clrscr();  
    s.show_grandfather();  
    s.show_father();  
}
```

```
s.show_son();
getch();
}
```

Output:

```
I am Grandfather
I am Father
I am Son
```

◆ 5. Hybrid Inheritance

- A **combination of two or more types** of inheritance.

```
class A { }; // base class
class B : public A { }; // derived from A
class C : public A { }; // derived from A
class D : public B, public C { }; // derived from B and C
```



💡 Notes say hybrid = hierarchical + multilevel; the example above is really hierarchical + multiple. Class D gets two copies of A; to avoid this, C++ uses **virtual inheritance** (class B : virtual public A).

👨👩👦 Family Example: Person, Father, Mother and Son (Hybrid)

```
#include<iostream.h>
#include<conio.h>

class Person // base class
{
public:
    void show_person()
    {
        cout << "I am a Person" << endl;
    }
};

class Father : virtual public Person // virtual: only one copy of Person
{
public:
    void show_father()
    {
        cout << "I am Father" << endl;
    }
};

class Mother : virtual public Person
{
public:
    void show_mother()
    {
        cout << "I am Mother" << endl;
    }
};

class Son : public Father, public Mother // derived from Father and Mother
```

```

{
public:
    void show_son()
    {
        cout << "I am Son" << endl;
    }
};

void main()
{
    Son s;
    clrscr();
    s.show_person();
    s.show_father();
    s.show_mother();
    s.show_son();
    getch();
}

```

Output:

```

I am a Person
I am Father
I am Mother
I am Son

```

2.6 Constructors

- A **constructor** is a special member function that is **called automatically when an object is created**.
- It is used to **initialize the data members** of a new object.
- It has the **same name as the class**.

 A constructor is like the setup done at a birthday party before guests arrive: it runs automatically at the start.

◆ Types of Constructors

1. Default Constructor

- **Compiler-defined:** made implicitly by the compiler when you write no constructor.
- **User-defined:** a constructor with **no arguments** (or default values for every argument). If the programmer does not write it, the compiler provides it.

2. Parameterized Constructor

- Takes a **specific number of arguments**. It gives user-wanted values to the data members of different objects.

3. Copy Constructor

- Initializes a new object using **another object of the same class** that was created earlier.
- Two kinds: **Implicit** (compiler-defined) and **Explicit** (user-defined).

◆ Special Characteristics of Constructors

- 1. They are **called automatically** when objects are created.
- 2. Objects are initialized before use.
- 3. They should be declared in the **public** section.
- 4. They have **no return type** (not even `void`).
- 5. They **cannot be inherited**, but a derived class can call the base class constructor.

- 6. They **cannot be static**.
- 7. Default and copy constructors are generated by the compiler when needed, and they are public.
- 8. They can have **default arguments**.
- 9. A constructor can call member functions of its class.
- 10. An object of a class with a constructor cannot be a member of a union (old C++ rule).

◆ Calling a Constructor

- Two ways: **1) Implicit call** and **2) Explicit call**.

```
#include<iostream.h>
#include<conio.h>
class abc
{
    int x, y;
public:
    abc()                // default constructor (user defined)
    {
        x = 10; y = 20;
        cout << "Default constructor is called\n";
    }
    void showdata()
    {
        cout << "x = " << x << endl;
        cout << "y = " << y << endl;
    }
};

void main()
{
    clrscr();
    abc a;                // implicit call
    abc b, c;
    abc d = abc();        // explicit call
    a.showdata();
    getch();
}
```

Output:

```
Default constructor is called
Default constructor is called
Default constructor is called
Default constructor is called
x = 10
y = 20
```

- The message is printed 4 times (for a, b, c, d); then `x = 10` and `y = 20`.

◆ Parameterized Constructor and Constructor Overloading

- **Constructor overloading** = many constructors in one class with **different number or type of parameters**.

```
#include<iostream.h>
#include<conio.h>
class abc
{
    int x, y;
public:
    abc()                // default constructor
    {
        x = 10; y = 20;
```

```

        cout << "Default constructor is called\n";
    }
    abc(double no1)           // parameterized (double)
    {
        x = no1; y = no1 + 5;
    }
    abc(int no1)              // parameterized (int)
    {
        x = no1; y = no1 + 5;
    }
    abc(int no1, int no2)     // parameterized (two int)
    {
        x = no1; y = no2;
    }
    void showdata()
    {
        cout << "x = " << x << endl;
        cout << "y = " << y << endl;
    }
};

void main()
{
    clrscr();
    abc a;                   // default constructor
    abc c(10);               // single int parameter
    abc d(10.50);            // single double parameter
    abc e(10, 12);           // two int parameters
    a.showdata();
    c.showdata();
    d.showdata();
    e.showdata();
    getch();
}

```

Output:

```

Default constructor is called
x = 10
y = 20
x = 10
y = 15
x = 10
y = 15
x = 10
y = 12

```

◆ Copy Constructor

- A member function that **initializes an object using another object of the same class**.
- Form: `classname(classname &)`

Type	Meaning
Implicit (compiler defined)	Used when you copy one object to another and the class has no copy constructor
Explicit (user defined)	Used when the class has a copy constructor written by the user

Implicit copy constructor

```

#include<iostream.h>
#include<conio.h>
class abc

```

```

{
public:
    int x;
    int y;
};

void main()
{
    clrscr();
    abc a;
    a.x = 10; a.y = 20;
    abc b(a);           // compiler defined copy constructor
    cout << b.x << "\n" << b.y << endl;
    getch();
}

```

Output:

```

10
20

```

Explicit copy constructor

```

#include<iostream.h>
#include<conio.h>
class abc
{
public:
    int x;
    int y;
    abc() { }
    abc(abc &a1)           // user defined copy constructor
    {
        x = a1.x + 5;
        y = a1.y + 3;
        cout << "User Defined Copy Constructor called\n";
    }
};

void main()
{
    clrscr();
    abc a;
    a.x = 10; a.y = 20;
    abc b = a;             // calls user defined copy constructor
    abc c(b);              // calls user defined copy constructor
    abc d;
    d = a;                 // does NOT call copy constructor (it is assignment)
    getch();
}

```

Output:

```

User Defined Copy Constructor called
User Defined Copy Constructor called

```

◆ Constructor with Default Arguments

```

#include<iostream.h>
#include<conio.h>

class add
{

```

```

private:
    int num1, num2, num3;
public:
    add(int = 0, int = 0);    // default argument constructor
    void sum();
    void display();
};

add::add(int n1, int n2)
{
    num1 = n1;
    num2 = n2;
    num3 = 0;
}

void add::sum()
{
    num3 = num1 + num2;
}

void add::display()
{
    cout << "\n The Sum of two number is " << num3;
}

void main()
{
    add obj1, obj2(5), obj3(10, 20);
    clrscr();
    obj1.sum();
    obj2.sum();
    obj3.sum();
    obj1.display();
    obj2.display();
    obj3.display();
    getch();
}

```

Output:

```

The Sum of two number is 0
The Sum of two number is 5
The Sum of two number is 30

```

- Objects can be made with no value, one value or two values: `add obj1, obj2(5), obj3(10, 20);`

Object	num1	num2	num3
obj1	0	0	0
obj2(5)	5	0	0
obj3(10, 20)	10	20	0

-  **Ambiguity:** if you write both `add::add() {}` and the default-argument constructor, then `add obj;` is **ambiguous** (the compiler does not know which one to call). Avoid this.

Destructor

- A **destructor** is a member function called **automatically when an object is destroyed**. It is the **last function** called before the object is destroyed.
- Syntax: `~name_of_the_class() { }` (class name with a **~ tilde** in front).

- It has **no parameters** and **returns nothing**.
- A class can have **only one destructor**; it **cannot be overloaded**.
- It can be defined outside the class.

```
#include<iostream.h>
#include<conio.h>
class add
{
private:
    int num1, num2, num3;
public:
    add(int = 0, int = 0);    // default argument constructor
    void sum();
    void display();
    ~add();                  // destructor
};

add::add(int n1, int n2)
{
    num1 = n1; num2 = n2; num3 = 0;
}

void add::sum()
{
    num3 = num1 + num2;
}

void add::display()
{
    cout << "\nsum of two number is : " << num3;
}

add::~~add()                // called automatically at end of program
{
    num1 = num2 = num3 = 0;
    cout << "\nAfter the final execution, the object has entered in the destructor to destroy object";
}

void main()
{
    clrscr();
    add obj1, obj2(5), obj3(10, 20);    // objects created and initialized
    obj1.sum();
    obj2.sum();
    obj3.sum();
    cout << "\n obj 1 : ";
    obj1.display();
    cout << "\n obj2 : ";
    obj2.display();
    cout << "\n obj 3:";
    obj3.display();
    getch();
}
```

Output:

```
obj 1 :
sum of two number is :0
obj2 :
sum of two number is :5
obj 3:
sum of two number is :30
```

After the final execution, the object has entered in the destructor to destroy object
 After the final execution, the object has entered in the destructor to destroy object
 After the final execution, the object has entered in the destructor to destroy object

- Sums shown: 0, 5, 30. Then the destructor message appears 3 times.

◆ Special Characteristics of Destructors

- 1. A destructor is a special member function that **destroys the objects** created by the constructor.
- 2. Same name as the class, with ~ **(tilde)** before it.
- 3. **Only one** destructor is possible.
- 4. It **cannot be overloaded**.
- 5. It needs **no argument** and returns **no value**.
- 6. It is called automatically when an object **goes out of scope**.
- 7. It **releases the memory** used by the objects.
- 8. Objects are destroyed in the **reverse order** of creation.

Constructor	Destructor
Creates and initializes the object	Destroys the object
Same name as class	Same name as class with ~
Can take arguments; can be overloaded	No arguments; cannot be overloaded
Called when object is created	Called when object is destroyed

★ Unit 2 Quick Revision

Topic	Remember
Access specifiers	public = everywhere, private = inside class, protected = class and derived classes
enum	Names for integer constants, starts from 0
Encapsulation	Wrap data and functions in a class
Abstraction	Show only essential details
Data hiding	Done with <code>private</code>
Friend function	Outside the class but can use private and protected members
Inheritance	Reuse of code, "is-a" relationship; base (parent) and derived (child)
Types	Single, Multiple, Hierarchical, Multilevel, Hybrid
Not inherited	Constructors, destructors, copy constructors, overloaded operators, friend functions
Constructor	Same name as class, no return type, called automatically
Constructor types	Default, Parameterized, Copy
Destructor	<code>~ClassName()</code> , one only, no arguments, reverse order of creation

UNIT 3 - Polymorphism

3.1 Concepts of Polymorphism

- **Polymorphism** = an entity (function or object) can have **more than one form**.
- The word comes from Greek and means "**having multiple forms**".
- The different forms come because the same entity gets **different meanings** in different places.

 One person can be a son, a father and a teacher at the same time. Same person, different roles.

3.2 Compile Time and Run Time Polymorphism

- **Compile time polymorphism:** the function is chosen by matching the **type and number of arguments**, and the compiler decides it **at compile time**. It is done by **function overloading** and **operator overloading**. Also called **static binding** or **early binding**.
- **Run time polymorphism:** the function is chosen **at run time**. It is done by **method overriding** (with virtual functions). Also called **dynamic binding** or **late binding**.

Compile time polymorphism	Run time polymorphism
Function to call is known at compile time	Function to call is known at run time
Also called overloading, early binding, static binding	Also called overriding, dynamic binding, late binding
Same name, different number or type of parameters	Same name, same number and type of parameters
Done by function overloading and operator overloading	Done by virtual functions and pointers
Fast execution	Slow execution
Less flexible	More flexible

3.3 Overloading and Overriding

◆ Function Overloading

- Two or more functions with the **same name but different parameters** (different types or different number of arguments).
- The compiler tells them apart only by these differences.
- It is done **within a single class**.
- You **cannot overload** functions that differ **only by return type**.
- **Advantage:** better readability; no need for different names for the same action.

```
#include<iostream.h>
#include<conio.h>
class sum
{
public:
    void addition(int x, int y)
    {
        cout << x + y << endl;
    }
    void addition(int x, double y)
    {
        cout << x + y << endl;
    }
}
```

```

    }
    void addition(int x, int y, int z)
    {
        cout << x + y + z << endl;
    }
};

void main()
{
    clrscr();
    sum s1;
    s1.addition(10, 20.50);    // output 30.5
    s1.addition(10, 15);      // output 25
    s1.addition(10, 5, 7);    // output 22
    getch();
}

```

Output:

```

30.5
25
22

```

Advantages of Function Overloading

- Improves **readability** and **reusability** of code.
- Gives consistency and easy code maintenance.
- Gives **flexibility**; one name can do different operations, so we do not need different names.

Disadvantages of Function Overloading

- Functions that differ **only in return type cannot be overloaded**.
- For a **static function**, the same parameters cannot be overloaded.

◆ Operator Overloading

- Giving **extra meaning to operators** like `+` `-` `*` `/` `=` , for user-defined types. By default they work only on built-in types (int, float, char).
- Example: `+` can add numbers and also join strings.
- It is a type of polymorphism: an operator gets a **user-defined meaning**.

Operator function vs normal function

- An operator function is like a normal function. The only difference: its name is always the keyword **operator** followed by the **operator symbol**, and it is called when that operator is used.
- Syntax: `return_type operator symbol (arguments) { body }`

Operators that CANNOT be overloaded

Operator	Name
<code>::</code>	Scope resolution operator
<code>sizeof</code>	Size of operator
<code>.</code>	Member selector
<code>.*</code>	Member pointer selector
<code>?:</code>	Ternary (conditional) operator

Implementing operator overloading – the operator function can be a:

- 1. **Member function**
- 2. **Non-member function**

- **3. Friend function**

- It can be a **member function** if the **left operand is an object of that class**. If the left operand is different, it must be a **non-member** function.
- It can be a **friend function** if it needs access to the **private and protected** members of the class.

Restrictions on Operator Overloading

- 1. **Precedence and associativity** of an operator cannot be changed.
- 2. **Number of operands** cannot be changed (unary stays unary, binary stays binary).
- 3. **No new operators** can be created; only existing operators can be overloaded.
- 4. You **cannot change how built-in types work** (you cannot change how integers are added).

◆ Overloading Arithmetic Operators

- Almost all arithmetic operators can be overloaded for user-defined types. Below, `+` and `*` are overloaded for two `Complex` objects.

```
#include<iostream.h>
#include<conio.h>
class Complex
{
private:
    int x, y;
public:
    Complex() { }
    Complex(int a, int b)
    {
        x = a;
        y = b;
    }
    Complex operator+(Complex obj)
    {
        Complex res;
        res.x = x + obj.x;
        res.y = y + obj.y;
        return res;
    }
    Complex operator*(Complex obj)
    {
        Complex res;
        res.x = x * obj.x;
        res.y = y * obj.y;
        return res;
    }
    void print()
    {
        cout << "X = " << x << endl << "Y = " << y << endl;
    }
};

void main()
{
    clrscr();
    Complex c1(10, 5);
    Complex c2(2, 4);
    Complex c3 = c1 + c2;        // calls operator+
    c3.print();                  // X = 12, Y = 9
    c3 = c1 * c2;                // calls operator*
    c3.print();                  // X = 20, Y = 20
    getch();
}
```

Output:

```
X = 12
Y = 9
X = 20
Y = 20
```

◆ Overloading Relational Operators

- Relational operators like `==` `!=` `>=` `<=` can be overloaded to **compare two objects**.

```
#include<iostream.h>
#include<conio.h>
class Time
{
    int hr, min, sec;
public:
    Time()
    {
        hr = 0; min = 0; sec = 0;
    }
    Time(int h, int m, int s)
    {
        hr = h; min = m; sec = s;
    }
    friend int operator==(Time &t1, Time &t2);    // overloading == operator
};

int operator==(Time &t1, Time &t2)              // operator function
{
    return (t1.hr == t2.hr && t1.min == t2.min && t1.sec == t2.sec);
}

void main()
{
    clrscr();
    Time a(3, 15, 45), b(3, 15, 45);
    if (a == b)
        cout << "Equal";
    else
        cout << "Not equal";
    getch();
}
```

Output:

```
Equal
```

◆ Copy Constructor vs Assignment Operator

Copy constructor	Assignment operator
Initializes a new object from an existing object	Copies values from one object to an already existing object
Called when the object is being created	Called after both objects already exist

```
Time tm(3, 15, 45);    // tm object created and initialized

Time t1(tm);           // copy constructor: t1 is created using tm

Time t2;               // t2 created first
t2 = tm;               // assignment operator: t2 gets values of tm
```

👉 Pointer to Base Class

- A **pointer to a derived class is type-compatible with a pointer to its base class**. Polymorphism uses this feature.
- Example: X is a base class and Y is derived from X. A pointer to X can also point to Y.

```
class X { /* class body */ };
class Y : public X { /* class body */ };

int main()
{
    X *ptr;          // pointer to base class
    X x;
    Y y;
    ptr = &x;         // pointing to base class object
    ptr = &y;         // pointing to derived class object using base class pointer
    return 0;
}
```

🔄 Dynamic Polymorphism (Overriding)

- A function shows **dynamic polymorphism** when it exists in more than one form and the call is decided **at run time**.
 - **Late binding** = the function is chosen at run time, not compile time. It makes the program more flexible.
- ◆ **Function Overriding**
- If a derived class **writes again a function of the base class**, that function is **overridden**. This is **function overriding**.

Requirements for Overriding

- 1. **Inheritance** is needed. Overriding cannot be done inside one class; we need a base class and a derived class.
- 2. The function must have **exactly the same declaration** in base and derived class: same name, same return type, same parameter list.

◆ Function Call Binding with Class Objects

- **Binding** = connecting a function call to the function body.
- If it is done **before the program runs**, it is **Early Binding / Static Binding / Compile-time Binding**.

```
#include<iostream.h>
#include<conio.h>
class Base
{
public:
    void show()
    {
        cout << "Base class\t";
    }
};

class Derived : public Base
{
public:
    void show()
    {
        cout << "Derived Class";
    }
}
```

```

    }
};

void main()
{
    clrscr();
    Base b;           // base class object
    Derived d;        // derived class object
    b.show();         // early binding
    d.show();         // early binding
    getch();
}

```

Output:

```
Base class    Derived Class
```

- Output: Base class Derived Class
- A base class object calls the **base version**; a derived class object calls the **derived version**.

◆ Function Call Binding using Base Class Pointer

- When a **base class pointer holds a derived class object**, the result can be unexpected.

```

#include<iostream.h>
#include<conio.h>

class Base
{
public:
    void show()
    {
        cout << "Base class";
    }
};

class Derived : public Base
{
public:
    void show()
    {
        cout << "Derived Class";
    }
};

void main()
{
    Base *b;           // base class pointer
    Derived d;         // derived class object
    clrscr();
    b = &d;            // base pointer points to derived object
    b->show();          // early binding happens
    getch();
}

```

- Output: Base class
- Even though the object is of the Derived class, the **Base class function** is called. The compiler only sees the base class pointer, so it fixes the call to the base function (**early binding**).

3.5 Virtual Function

- A **virtual function** is a function in the **base class**, overridden in the derived class, which tells the compiler to do **late binding** on it.
- The **virtual** keyword makes a base class function virtual.

Problem without virtual : a base pointer holding a derived object always calls the **base version** (output: Base class).

Using virtual :

```
#include<iostream.h>
#include<conio.h>

class Base
{
public:
    virtual void show()          // virtual function
    {
        cout << "Base class";
    }
};

class Derived : public Base
{
public:
    void show()
    {
        cout << "Derived Class";
    }
};

void main()
{
    Base *b;          // base class pointer
    Derived d;        // derived class object
    clrscr();
    b = &d;
    b->show();        // late binding happens
    getch();
}
```

- Output: Derived Class . With **virtual** , late binding happens and the **derived version** is called.

Important points to remember

- 1. Only the **declaration** in the base class needs the **virtual** keyword, not the definition.
- 2. If a function is virtual in the base class, it is **virtual in all derived classes**.

◆ Pure Virtual Function

- A pure virtual function has **= 0** in its declaration and **no body**.

```
class SomeClass
{
public:
    virtual void pure_virtual() = 0;    // a pure virtual function
};
```

Full program with output:

```
#include<iostream.h>
#include<conio.h>
```

```

class Shape                                // abstract class
{
public:
    virtual void draw() = 0;    // pure virtual function
};

class Circle : public Shape
{
public:
    void draw()
    {
        cout << "Drawing Circle" << endl;
    }
};

class Square : public Shape
{
public:
    void draw()
    {
        cout << "Drawing Square" << endl;
    }
};

void main()
{
    Shape *s;
    Circle c;
    Square q;
    clrscr();
    s = &c;
    s->draw();
    s = &q;
    s->draw();
    getch();
}

```

Output:

```

Drawing Circle
Drawing Square

```

- `= 0` is called the **pure specifier**. It does not assign a value; it only shows the function is pure virtual.
- A class with **at least one pure virtual function** is an **abstract class**.
- You **cannot create an object** of an abstract class: `SomeClass s1;` gives an **error**.
- A class derived from an abstract class **must override** the pure virtual function; otherwise it also becomes abstract.

Static vs Dynamic Polymorphism

- Static polymorphism is more **efficient**; dynamic polymorphism is more **flexible**.
- **Static (early) binding**: method bound to its call at **compile time**; no run-time search is needed.
- **Dynamic (late) binding**: bound at **run time**; this needs an extra step of searching the function at run time.
- As programs grow, **flexibility** is needed. With dynamic binding, calls are resolved at run time, so the call can change **without changing the code**.

VS Function Overloading vs Function Overriding

Point	Function Overloading	Function Overriding
Definition	Two or more functions with the same name but different parameters	One function in the parent class and another in the child class with the same name and parameters
Signature	Must differ in number or type of parameters	Must not change
Behaviour	Defines several behaviours	Changes the behaviour of the base function
Scope	Same class	Two classes (base and derived)
Inheritance	Not needed	Needed
Polymorphism	Compile time	Run time

★ Unit 3 Quick Revision

Topic	Remember
Polymorphism	One name, many forms
Compile time	Overloading, early/static binding, fast
Run time	Overriding, late/dynamic binding, virtual functions, more flexible
Function overloading	Same name, different parameters, same class; return type alone is not enough
Operator overloading	operator keyword + symbol
Not overloaded	::, sizeof, ., .*, ?:
Restrictions	Cannot change precedence, number of operands, or create new operators
Operator function types	Member, non-member, friend
Copy constructor vs assignment	New object vs existing object
Overriding needs	Inheritance + same declaration
Without virtual	Base pointer calls base function (early binding)
With virtual	Base pointer calls derived function (late binding)
Pure virtual	= 0 ; class becomes abstract; no object possible



UNIT 4 - Data Structure (Stack)



Data Structure Introduction

- **Data** = a collection of facts and figures, or values in a particular format.
- A **data item** = a single set of values.
- Data items can be **elementary** or **group**:
 - **Elementary data item**: cannot be divided further. Example: ID, Age, Gender.
 - **Group data item**: divided into sub-items. Example: **Name** (first, middle, last name) and **Address** (street, area).



What is a Data Structure?

- A **data structure** is a **specific way to store and organize data in computer memory** so that it can be **used efficiently** later.
- The logical or mathematical model of a particular organization of data is called a data structure.
- A good data structure must satisfy two things:
 - It must be **rich enough** to show the real relationships of the data with the real-world object.
 - It must be **simple enough** so the data can be processed efficiently every time.



A bookshelf with labels is a data structure: books are arranged so you can find them quickly.



Categories of Data Structure

- Two major types: **1. Linear** and **2. Non-linear**.

◆ Linear Data Structure

- Elements form a **specific order**, one after the other.
- Two ways to keep a linear structure in memory:
 - Using **linear memory locations**: these are **arrays**.
 - Using **pointers or links**: these are **linked lists**.
- Examples: **Arrays, Queues, Stacks, Linked lists**.

◆ Non-linear Data Structure

- Used for data that has a **hierarchical relationship** among elements.
- Examples: **Graphs, the family of trees, table of contents**.
- **Tree**: data has a hierarchical relationship; it is shown as a **rooted tree**.
- **Graph**: data has a relationship between **pairs of elements**, not necessarily hierarchical.

Linear	Non-linear
Elements in a sequence, one after another	Elements in a hierarchy or connected pairs
Array, Stack, Queue, Linked list	Tree, Graph



What is a Stack?

- A **stack** is a **linear data structure** in which **all insertion and deletion happen at one end only**, called the **TOP**.
- It can be made using **arrays**.
- It is used to **convert and evaluate expressions** in Polish notations: **Infix, Prefix, Postfix**.

- Because the **last item added is the first item removed**, a stack follows **LIFO (Last In First Out)**. It is also called a **Pile** or **Push Down List**.

 A pile of plates: you add a plate on top and take a plate from the top. A tray holder in a cafeteria or books in a box open at one end are also stacks.

```

      insert / delete here
          ↓
TOP → |  30  |
      |  20  |
      |  10  |
      +-----+

```

◆ Stack as an Abstract Data Type (ADT)

- A stack of elements of one type is a finite sequence of elements with these operations:
 - Initialize** the stack to be empty.
 - Check whether the stack is **empty**.
 - Check whether the stack is **full**.
 - If not full, **insert** a new node at the top: **PUSH** operation.
 - If not empty, **retrieve** the node at the top.
 - If not empty, **delete** the node at the top: **POP** operation.

⚙ Stack Operations

Operation	Meaning
PUSH	Insert an element into the stack
POP	Delete an element from the stack
PEEP	Get the value of a specified element from the top of the stack
CHANGE	Change the value of a specified element from the top of the stack

- Overflow** = trying to push into a full stack. **Underflow** = trying to pop from an empty stack.

+ Algorithm to Insert (PUSH)

- Main tasks: **check overflow, increment TOP, insert the element**.
- PUSH (S, TOP, ITEM)
 - S** = array representing the stack
 - TOP** = position of the topmost element
 - ITEM** = value to be inserted
 - MAX** = maximum number of elements the stack can hold

```

Step 1: [Check for Stack Overflow]
        If TOP >= MAX-1 then
            Write ("Stack Overflow")
            Exit
        [End If]
Step 2: [Increment TOP]
        TOP := TOP + 1
Step 3: [Insert Element]
        S[TOP] := ITEM
Step 4: [Finished]
        Return

```

— Algorithm to Delete (POP)

- Main tasks: **check underflow, decrement TOP**.
- POP (S, TOP)

```
Step 1: [Check for underflow on stack]
        If TOP < 0 then
            Write ("Stack Underflow on POP")
            Exit
        [End If]
Step 2: [Decrement pointer]
        TOP := TOP - 1
Step 3: [Return top element of stack]
        Return (S[TOP + 1])
```

👁️ Algorithm to Get the Ith Element from the Top (PEEP)

- Gets the value **without deleting it**. Main tasks: **check underflow, retrieve the element**.
- PEEP (S, TOP, I) where **I** = position of the element from the top.

```
Step 1: [Check for Stack Underflow]
        If TOP - I + 1 <= -1 then
            Write ("Stack Underflow on PEEP")
            Exit
        [End of If]
Step 2: [Return Ith value from top of stack]
        Return (S[TOP - I + 1])
```

✏️ Algorithm to Change the Ith Element from the Top (CHANGE)

- CHANGE (S, TOP, ITEM, I) where **ITEM** = new value of the Ith element from the top.

```
Step 1: [Check for Stack Underflow]
        If TOP - I + 1 <= -1 then
            Write ("Stack Underflow on change")
            Exit
Step 2: [Change Ith element from top of Stack]
        S[TOP - I + 1] := ITEM
Step 3: [Finished]
        Exit
```

💻 Representation of Stack using Arrays

- A stack in memory uses a **linear array STACK** and a **pointer variable top** that holds the position of the top element.
- Initially **top = -1** (stack is empty).

```
#include<iostream.h>
#include<conio.h>
#include<stdlib.h>

class stack
{
    int stk[5];
    int top;
public:
    stack()
    {
        top = -1;
    }
};
```

```

}
void push(int x)
{
    if (top >= 4) {
        cout << "stack overflow";
        return;
    }
    top = top + 1;
    stk[top] = x;
    cout << "inserted " << x;
}
void pop()
{
    if (top < 0) {
        cout << "stack underflow";
        return;
    }
    cout << "deleted " << stk[top--];
}
void display();
void peep();
void change();
};

void stack::display()
{
    if (top < 0) {
        cout << " stack empty";
        return;
    }
    for (int i = top; i >= 0; i--)
        cout << stk[i] << " ";
}

void stack::peep()
{
    int i;
    cout << "enter the position";
    cin >> i;
    if (top < 0) {
        cout << " stack empty";
        return;
    }
    cout << stk[top - i + 1];
}

void stack::change()
{
    int i, item;
    cout << "\n enter the position to enter";
    cin >> i;
    cout << "\n Enter the item to be insert :";
    cin >> item;
    if (top < 0) {
        cout << " stack empty";
        return;
    }
    stk[top - i + 1] = item;
}

int main()
{
    clrscr();
    int ch;

```

```

stack st;
while (1)
{
    cout << "\n1.push 2.pop 3.display 4.peep 5.change 6.exit\nenter ur choice: ";
    cin >> ch;
    switch (ch)
    {
        case 1:
            cout << "enter the element: ";
            cin >> ch;
            st.push(ch);
            break;
        case 2:
            st.pop();
            break;
        case 3:
            st.display();
            break;
        case 4:
            st.peep();
            break;
        case 5:
            st.change();
            break;
        case 6:
            exit(0);
    }
}
}

```

Output (sample run: push 10, 20, 30, display, pop, display, peep position 1, change position 2 to 99, display, exit):

```

1.push 2.pop 3.display 4.peep 5.change 6.exit
enter ur choice: 1
enter the element: 10
inserted 10
enter ur choice: 1
enter the element: 20
inserted 20
enter ur choice: 1
enter the element: 30
inserted 30
enter ur choice: 3
30 20 10
enter ur choice: 2
deleted 30
enter ur choice: 3
20 10
enter ur choice: 4
enter the position1
20
enter ur choice: 5
enter the position to enter2
Enter the item to be insert :99
enter ur choice: 3
20 99
enter ur choice: 6

```

💡 This program is in **Turbo C++** style (`iostream.h` , `conio.h` , `clrscr()`), same as your notes. In a modern compiler use `#include<iostream>` with `using namespace std;` and remove `conio.h` and `clrscr()` .



Stack Application

- Two types of stack applications: **(1) Polish expression** and **(2) Recursion**.
- Some computers also do stack operations at the **hardware (machine) level** (stack machines).

◆ Polish Expression

- When operators are written **after the operands**, it is called **Reverse Polish Notation**.
- Three types of expressions: **1) Infix, 2) Prefix, 3) Postfix**.
- "Pre", "Post" and "In" tell the **position of the operator** relative to the two operands.

Notation	Operator position	Example
Infix	Between the operands	A + B
Prefix	Before the operands	+ A B
Postfix	After the operands	A B +

◆ Infix Notation

- Operators are placed **between the operands**. Example: A + (B * C)
- The order of operations is decided by **parentheses** or **operator precedence**.
- Levels of precedence (highest to lowest):

Priority	Operator
1	(
2	^, \$
3	%
4	/ or * (left to right)
5	+ or - (left to right)
6	)



Algorithm to Convert Infix to Postfix

- We use an **operator stack** (works as FILO, First In Last Out), an **output** (works as FIFO) and one **input string**.
- Steps:

- 1) Scan the expression from left to right.
- 2) If it is an operand, put it directly into the output.
- 3) If it is an operator and the operator stack is empty, push it onto the stack.
- 4) If the incoming operator has HIGHER precedence than the top of the stack, push it onto the stack.
- 5) If the incoming operator has LOWER precedence than the top of the stack, pop the top into the output, then compare again with the new top.
- 6) If the incoming operator has EQUAL precedence with the top, use associativity:
- 7) left to right -> pop the top into output, then push the incoming operator.
- 8) right to left -> push the incoming operator.
- 9) If the scanned character is '(', push it onto the stack.
- 10) If the scanned character is ')', pop operators into output until '(' is found, and discard both brackets.
- 11) Repeat steps 2 to 10 until the whole expression is scanned.
- 12) At the end, pop all remaining operators from the stack into the output.
- 13) The output is the postfix expression.

Never pop (or) into the output just because of precedence.

Example: Convert $A+(B+C*D)+E$ to Postfix

No.	Symbol	Operator stack	Postfix output
1	A	(empty)	A
2	+	+	A
3	(	+ (	A
4	B	+ (	A B
5	+	+ (+	A B
6	C	+ (+	A B C
7	*	+ (+ *	A B C
8	D	+ (+ *	A B C D
9	)	+	A B C D * +
10	+	+	A B C D * + +
11	E	+	A B C D * + + E
12	end	(empty)	A B C D * + + E +

- **Postfix:** $ABCD*++E+$

⚠ Your notes show the answer $ABCD*++E$ and start the trace with $*$. That is a mistake (the expression has no $*$ before $($). The correct postfix is $ABCD*++E+$ as above.

Algorithm to Convert Infix to Prefix

- Same idea as postfix, but:

- 1) Reverse the infix expression (and swap '(' with ')').
- 2) Convert the reversed expression to postfix style using the operator stack.
- 3) Reverse the result. This is the prefix expression.

- Scan from **right to left**. Operands go straight to output. For equal precedence use the associativity rules. A $)$ is pushed and a $($ pops until the matching bracket.

Example: Convert $(A+B*C)$ to Prefix

- Reverse the expression: $(C*B+A)$

No.	Symbol	Operator stack	Output
1	(	(	
2	C	(	C
3	*	(*	C
4	B	(*	C B
5	+	(+	C B *
6	A	(+	C B * A
7	)	(empty)	C B * A +

- The result is `CB*A+`. **Reverse it** to get the prefix: `+A*BC`

VS Difference between Stack and Queue

Stack	Queue
LIFO (Last In First Out)	FIFO (First In First Out)
The element inserted first is accessed last (only one open end)	The element inserted first is served first
Has one open end ; the latest element is reached using top	Has two open ends : rear and front
Insert with push()	Insert with push() (enqueue) at the rear end
Delete with pop()	Delete with pop() (dequeue) from the front end
Example: plates placed one above the other	Example: queue for a bus

- **Stack**: insertion and deletion at **one side only (top)**. Insertion = **push**, deletion = **pop**. A pointer **top** keeps track of the last element.
- **Queue**: insertion only at **rear**, deletion only at **front**. Insertion = **enqueue**, deletion = **dequeue**. Two pointers, **front** and **rear**, are kept.

🔄 Recursion as an Application of Stack

- A **function that calls itself** is called **recursion**. (First function calls the second, the second calls the third, and so on.)
- Recursion is an important concept in computer science.
- Each call **pushes** the current variables, arguments and return address onto a **stack**. When the call finishes, they are **popped** back.

🧸 Recursion is like Russian nesting dolls: open one doll and you find a smaller doll inside, until the smallest one (the base case).

- Notes: recursion takes extra time and memory because all current variables and objects are pushed onto the stack. Its benefit is **short and simple code** for problems like factorial.

♦ General Algorithm for Recursion

```

Step 1: [Preface]
        Save the argument and the return address.
Step 2: [Body of function]
        If the base criterion is met, go to Step 3.
        Else do the partial computation and go to Step 1.
Step 3: [Last section]
        Restore the most recently saved parameters and return address.

```

♦ Algorithm of Recursive Factorial

- Given an integer **N**, find **N!**.
- A stack **A** stores an **activation record** for each recursive call. Each record has the current value of **N** and the current **return address**.
- When a function is called, a copy of the argument and address is **pushed**. When it finishes, they are **popped**.

```

1. [Save N and return address]
   Call push(A, N, ret_add)
2. [Is the base criterion found?]

```

```

    If N == 0 then
        Fact = 1
        goto step 4
    Else
        param = N - 1
        ret_add = step 3
        goto step 1
3. [Calculate N!]
    Fact = N * Fact
4. [Restore N and ret_add]
    tem_rec = pop(A)
    goto tem_rec

```

Program: Factorial using Recursion

```

#include<iostream.h>
#include<conio.h>

int factorial(int n);

void main()
{
    int n;
    cout << "Enter a positive integer: ";
    cin >> n;
    cout << "Factorial of " << n << " = " << factorial(n);
    getch();
}

int factorial(int n)
{
    if (n > 1)
        return n * factorial(n - 1);
    else
        return 1;
}

```

Output (sample run):

```

Enter a positive integer: 5
Factorial of 5 = 120

```

- Dry run for $n = 3$: $\text{factorial}(3) = 3 * \text{factorial}(2) = 3 * (2 * \text{factorial}(1)) = 3 * 2 * 1 = 6$

★ Unit 4 Quick Revision

Topic	Remember
Data structure	A way to store and organize data in memory for efficient use
Linear	Array, Stack, Queue, Linked list
Non-linear	Tree, Graph
Stack	Linear, insert and delete at top only, LIFO , also called Pile or Push Down List
Stack operations	PUSH, POP, PEEP, CHANGE
Overflow / Underflow	Push on full stack / pop from empty stack
PUSH steps	Check $\text{TOP} \geq \text{MAX}-1$, $\text{TOP} = \text{TOP}+1$, $\text{S}[\text{TOP}] = \text{ITEM}$
POP steps	Check $\text{TOP} < 0$, $\text{TOP} = \text{TOP}-1$, return $\text{S}[\text{TOP}+1]$

Topic	Remember
PEEP / CHANGE	Use $S[TOP - I + 1]$
Stack applications	Polish expression (infix, prefix, postfix) and recursion
Infix to postfix	Operand to output, operators with stack by precedence, <code>)</code> pops till <code>(</code>
Infix to prefix	Reverse, convert, reverse again
Stack vs Queue	LIFO vs FIFO; one end vs two ends
Recursion	Function calls itself; uses stack for arguments and return address



UNIT 5 - Data Structure (Queue)



What is a Queue?

- A **queue** is a **linear list** of elements. **Insertion** happens at one end called the **rear**, and **deletion** happens at the other end called the **front**.
- The element inserted **first** is deleted **first**. So a queue follows **FIFO (First In First Out)**.
- In this unit the queue is made using **arrays**.



People standing in line to buy a movie ticket: the first person in line gets the ticket first. Payment counter in a shopping mall is also a queue.

- Examples inside a computer: tasks waiting for the **line printer**, for **disk access**, or for the **CPU in a time-sharing system**.

```

delete here          insert here
(FRONT)              (REAR)
  ↓                  ↓
[ 10 ] [ 20 ] [ 30 ] [ 40 ] [   ] [   ]

```



Queue as an ADT (Abstract Data Type)

- An abstract data type needs: **1)** a particular way in which the components are related, and **2)** a statement of the **operations** that can be done on the elements.
- Queue as an ADT has these operations:
 - **Initialize** the queue to be empty.
 - Check whether the queue is **empty**.
 - Check whether the queue is **full**.
 - **Insert** a new element after the last element, if the queue is not full.
 - **Retrieve** the first element, if the queue is not empty.
 - **Delete** the first element, if the queue is not empty.



Definition and Concept

- Queue is a sub class of linear data structure. Elements are inserted from the **rear** and deleted from the **front**.
- Insertion and deletion happen at **two different ends**, so elements come out in the **same order** in which they went in. That is why it is a **FIFO list**.
- By the structure used to make it:
 - **Static queue:** made using a **fixed-length array**.
 - **Dynamic queue:** made using a **linked list**. (Only static queue is covered here.)
- By the way it works, there are three types:

Type	Meaning
Simple queue	Normal queue: insert at rear, delete at front
Circular queue	Last position is joined to the first position, so freed space is reused
Double ended queue (Deque)	Insertion and deletion are possible at both ends

Operations on Simple Queue

Operation	Meaning
Insert	Add an element at the rear
Delete	Remove an element from the front

- **Overflow** = inserting in a full queue. **Underflow** = deleting from an empty queue.
- At the start, `front = -1` and `rear = -1` (queue is empty).

+ Algorithm to Insert in Simple Queue

- QINSERT (Q, front, rear, size, Item)
 - **Q** = array for the queue, **front** = index of front element, **rear** = index where the next element goes, **size** = maximum elements, **Item** = value to insert.

```
Step 1: [Check for overflow]
        If rear = size then
            Output "Queue Overflow"
            Exit
Step 2: [When queue is initially empty]
        If front = -1 then
            front = 0
            rear = 0
Step 3: [Insert an element]
        Q[rear] = Item
Step 4: [Increment rear]
        rear = rear + 1
```

— Algorithm to Delete from Simple Queue

- QDELETE (Q, front, rear)

```
Step 1: [Check for underflow]
        If front = -1 then
            Output "Queue Underflow"
            Exit
        Else
            Item = Q[front]
Step 2: [Check if the queue has only one element]
        If front = rear - 1 then
            front = -1
            rear = -1
        Else
            front = front + 1
Step 3: Exit
```

Program: Simple Queue using Array

```
#include<iostream.h>
#include<conio.h>
#include<stdlib.h>

class queue
{
    int q1[5];
    int rear, front;
public:
    queue()
    {
        rear = -1;
        front = -1;
    }
};
```

```

}
void insert(int x)
{
    if (rear == 5)
    {
        cout << "queue over flow";
        return;
    }
    if (front == -1)
    {
        front = 0;
        rear = 0;
    }
    ql[rear] = x;
    rear++;
    cout << "inserted " << x;
}
void delet()
{
    if (front == -1)
    {
        cout << "queue under flow";
        return;
    }
    cout << "deleted " << ql[front];
    if (front == rear - 1)
    {
        front = -1;
        rear = -1;
    }
    else
        front++;
}
void display();
};

void queue::display()
{
    int i;
    if (front == -1)
    {
        cout << " queue empty";
        return;
    }
    for (i = front; i < rear; i++)
        cout << ql[i] << " ";
}

void main()
{
    int ch;
    queue qu;
    clrscr();
    while (1)
    {
        cout << "\n1.insert 2.delet 3.display 4.exit\nenter ur choice: ";
        cin >> ch;
        switch (ch)
        {
            case 1:
                cout << "enter the element: ";
                cin >> ch;
                qu.insert(ch);
                break;

```

```

        case 2:
            qu.delet();
            break;
        case 3:
            qu.display();
            break;
        case 4:
            exit(0);
    }
}
}

```

Output (sample run):

```

1.insert 2.delet 3.display 4.exit
enter ur choice: 1
enter the element: 10
inserted 10
1.insert 2.delet 3.display 4.exit
enter ur choice: 1
enter the element: 20
inserted 20
1.insert 2.delet 3.display 4.exit
enter ur choice: 1
enter the element: 30
inserted 30
1.insert 2.delet 3.display 4.exit
enter ur choice: 3
10 20 30
1.insert 2.delet 3.display 4.exit
enter ur choice: 2
deleted 10
1.insert 2.delet 3.display 4.exit
enter ur choice: 3
20 30
1.insert 2.delet 3.display 4.exit
enter ur choice: 4

```

Circular Queue

◆ Why Circular Queue?

- In a simple queue, memory is not used well. If a queue of size 5 gets 5 elements and then **2 are deleted**, we **still cannot insert** a new element until the whole queue is empty, though 2 places are free.
- In a **circular queue**, the **space of deleted elements is reused** for new elements.

Simple queue (size 5): after deleting 2 elements
 [--] [--] [30] [40] [50] rear is at the end -> "overflow" even though 2 places are free

Circular queue: rear goes back to the start
 [60] [--] [30] [40] [50] 60 is inserted in the free place

 Think of a merry-go-round: after the last seat, you come back to the first seat.

◆ Overflow Condition in Circular Queue

- Queue is full when **either** of these is true (F = front, R = rear, N = size):

Condition	Meaning
$F = 0$ and $R = N-1$	Front is at the first place and rear is at the last place
$F = R + 1$	Rear is just behind the front (rear has come around and met the front)

+ Algorithm to Insert in Circular Queue

- CQINSERT (Q, R, N, F, Item)
 - **Q** = array of the circular queue, **F** = front index, **R** = rear index, **N** = maximum elements, **Item** = value to insert.
 - Initially **F = -1** and **R = -1**.

```

Step 1: [Check for overflow]
        If (F = 0 and R = N-1) OR (F = R+1) then
            Write "queue overflow on insert"
            Exit
Step 2: [Update F and R according to need]
        [First element in the queue]
        If F = -1 then
            F := 0
            R := 0
        [Rear at the end of the circular queue]
        Else if R = N-1 then
            R := 0
        Else
            R := R + 1
        [End if]
Step 3: [Insert the element]
        Q[R] := Item
Step 4: Exit
  
```

— Algorithm to Delete from Circular Queue

```

Step 1: [Check underflow]
        If F = -1 then
            Write "queue underflow"
            Exit
        [End if]
Step 2: [Get the element to be deleted]
        Item := Q[F]
Step 3: [Reset F and R]
        [Deletion of the last element]
        If F = R then
            F := -1
            R := -1
        [Front at the end of the circular queue]
        Else if F = N-1 then
            F := 0
        Else
            F := F + 1
        [End if]
  
```

💡 Your notes write $F = N$ for the end position; with array index starting from 0 the last place is $N-1$, as used in the program below.

📄 Program: Circular Queue using Array

```

#include<iostream.h>
#include<conio.h>
#include<stdlib.h>
#define size 5
  
```

```

class cir
{
    int Q[size];
    int rear, front;
public:
    cir()
    {
        rear = -1;
        front = -1;
    }
    void insert()
    {
        int no;
        if ((front == 0 && rear == size - 1) || (front == rear + 1))
        {
            cout << "\n overflow";
            return;
        }
        else if (front == -1)
        {
            front = 0;
            rear = 0;
        }
        else if (rear == size - 1)
        {
            rear = 0;
        }
        else
            rear++;
        cout << "\n Enter element :";
        cin >> no;
        Q[rear] = no;
    }
    void del()
    {
        if (front == -1)
        {
            cout << "\n underflow ";
            return;
        }
        cout << "\n Element deleted:" << Q[front];
        if (front == rear)
        {
            front = -1;
            rear = -1;
        }
        else
        {
            if (front == size - 1)
                front = 0;
            else
                front++;
        }
    }
    void print()
    {
        int i;
        if (front == -1)
        {
            cout << "\n queue is empty ";
            return;
        }
        if (front <= rear)
    }

```

```

        {
            for (i = front; i <= rear; i++)
                cout << "\n" << Q[i];
        }
        else
        {
            for (i = front; i < size; i++)
                cout << "\n" << Q[i];
            for (i = 0; i <= rear; i++)
                cout << "\n" << Q[i];
        }
    }
};

void main()
{
    int ch;
    cir qu;
    clrscr();
    while (1)
    {
        cout << "\n1.insert 2.delet 3.display 4.exit\nenter ur choice: ";
        cin >> ch;
        switch (ch)
        {
            case 1:
                qu.insert();
                break;
            case 2:
                qu.del();
                break;
            case 3:
                qu.print();
                break;
            case 4:
                exit(0);
        }
    }
}

```

Output (sample run: insert 10, 20, 30, 40, 50, then 60 gives overflow; delete 2; insert 60; display):

```

1.insert 2.delet 3.display 4.exit
enter ur choice: 1
Enter element :10
... (20, 30, 40, 50 inserted the same way) ...
1.insert 2.delet 3.display 4.exit
enter ur choice: 1
overflow
1.insert 2.delet 3.display 4.exit
enter ur choice: 2
Element deleted:10
1.insert 2.delet 3.display 4.exit
enter ur choice: 2
Element deleted:20
1.insert 2.delet 3.display 4.exit
enter ur choice: 1
Enter element :60
1.insert 2.delet 3.display 4.exit
enter ur choice: 3
30
40

```

50
60

- Here 60 went into the **free place at the start** of the array, which a simple queue cannot do.

◆ Applications of Circular Queue

- CPU scheduling
- Memory management
- Traffic management

↔ Double Ended Queue (Deque)

- A deque is a queue in which **insertion and deletion can be done at either end or both ends**.
- Two variations:

Type	Meaning
Input restricted deque	Insertion only at one end ; deletion at both ends
Output restricted deque	Deletion only at one end ; insertion at both ends

- The algorithms below use the **circular queue** form of a deque.

```
insert / delete      insert / delete
  ↓↑                ↓↑
LEFT (front) [ 5 ] [ 10 ] [ 20 ] [ 30 ] RIGHT (rear)
```

+ Algorithm to Insert in Deque

- DQ = array holding the deque, DIR = side of insertion (Left or Right), N = size. (1-based positions as in the notes.)

```
Step 1: [Get the value of DIR]
        Read DIR
Step 2: [Check for overflow]
        If (Left = 1 and Right = N) or (Left = Right + 1) then
            Write "Overflow on insertion"
            Exit
        [End if]
Step 3: [Deque is empty?]
        If Left = Null then
            Left := 1
            Right := 1
            DQ[Left] := Item
            Exit
        [End if]
Step 4: [DIR is Left?]
        If DIR = "Left" then
            If Left = 1 then
                Left := N
            Else
                Left := Left - 1
            [End if]
            DQ[Left] := Item
Step 5: [DIR is Right?]
        Else If DIR = "Right" then
            If Right = N then
                Right := 1
            Else
                Right := Right + 1
            [End if]
            DQ[Right] := Item
```

```
[End if]
Step 6: Exit
```

— Algorithm to Delete from Deque

- **Item** holds the data of the deleted element.

```
Step 1: [Check for underflow]
    If Left = Null
        Output "Underflow on deletion"
        Exit
    [End if]
Step 2: [Deleting the last element]
    If Left = Right then
        Item := DQ[Left]
        Left := Null
        Right := Null
        Return (Item)
    [End if]
Step 3: Read DIR
Step 4: [DIR is Left?]
    If DIR = "Left" then
        Value := DQ[Left]
        If Left = N then
            Left := 1
        Else
            Left := Left + 1
        [End if]
Step 5: [DIR is Right?]
    Else If DIR = "Right" then
        Value := DQ[Right]
        If Right = 1 then
            Right := N
        Else
            Right := Right - 1
        [End if]
    [End if]
Step 6: Return (Value)
```

Program: Deque using Circular Array

```
#include<iostream.h>
#include<stdlib.h>
#include<conio.h>
#define MAX 5

class deq
{
private:
    int deque_arr[MAX];
    int front;
    int rear;
public:
    deq()
    {
        front = -1;
        rear = -1;
    }
    void insert_frontend(int item);
    void insert_rearend(int item);
    int delete_frontend();
    int delete_rearend();
    void display();
};
```

```

void deq::insert_frontend(int item)
{
    if (((front == 0) && (rear == MAX - 1)) || (front == rear + 1))
    {
        cout << "\nqueue overflow\n";
        return;
    }
    else if (front == -1)           // queue is empty
    {
        front = 0;
        rear = 0;
    }
    else if (front == 0)
        front = MAX - 1;
    else
        front = front - 1;
    deque_arr[front] = item;
}

void deq::insert_rearend(int item)
{
    if (((front == 0) && (rear == MAX - 1)) || (front == rear + 1))
    {
        cout << "\nqueue overflow\n";
        return;
    }
    else if (rear == -1)           // queue is empty
    {
        front = 0;
        rear = 0;
    }
    else if (rear == MAX - 1)
        rear = 0;
    else
        rear = rear + 1;
    deque_arr[rear] = item;
}

int deq::delete_frontend()
{
    int item;
    if (front == -1)
    {
        cout << "\nqueue underflow\n";
        return -1;
    }
    item = deque_arr[front];
    if (front == rear)           // only one element
    {
        front = -1;
        rear = -1;
    }
    else if (front == MAX - 1)
        front = 0;
    else
        front = front + 1;
    return item;
}

int deq::delete_rearend()
{
    int item;
    if (front == -1)

```

```

{
    cout << "\nqueue underflow\n";
    return -1;
}
item = deque_arr[rear];
if (front == rear)           // only one element
{
    front = -1;
    rear = -1;
}
else if (rear == 0)
    rear = MAX - 1;
else
    rear = rear - 1;
return item;
}

void deq::display()
{
    int i;
    if (front == -1)
    {
        cout << "\nqueue empty\n";
        return;
    }
    cout << "\nqueue elements :\n";
    i = front;
    if (front <= rear)
    {
        while (i <= rear)
            cout << " " << deque_arr[i++];
    }
    else
    {
        while (i <= MAX - 1)
            cout << " " << deque_arr[i++];
        i = 0;
        while (i <= rear)
            cout << " " << deque_arr[i++];
    }
    cout << "\n";
}

void main()
{
    deq q1;
    int choice, item;
    clrscr();
    while (1)
    {
        cout << "\n\n1.insert at the front end\n";
        cout << "2.insert at the rear end\n";
        cout << "3.delete from front end\n";
        cout << "4.delete from rear end\n";
        cout << "5.display\n";
        cout << "6.quit\n";
        cout << "\nenter your choice : ";
        cin >> choice;
        switch (choice)
        {
            case 1:
                cout << "\ninput the element for adding in queue : ";
                cin >> item;
                q1.insert_frontend(item);

```

```

        break;
    case 2:
        cout << "\ninput the element for adding in queue : ";
        cin >> item;
        q1.insert_rearend(item);
        break;
    case 3:
        cout << "\nelement deleted from front end is : " << q1.delete_frontend();
        break;
    case 4:
        cout << "\nelement deleted from rear end is : " << q1.delete_rearend();
        break;
    case 5:
        q1.display();
        break;
    case 6:
        exit(1);
    default:
        cout << "\nwrong choice\n";
    }
}
}

```

Output (sample run: insert 10 at front, 20 and 30 at rear, 5 at front, display, delete from front, delete from rear, display):

```

enter your choice : 1
input the element for adding in queue : 10
enter your choice : 2
input the element for adding in queue : 20
enter your choice : 2
input the element for adding in queue : 30
enter your choice : 1
input the element for adding in queue : 5
enter your choice : 5
queue elements :
5 10 20 30
enter your choice : 3
element deleted from front end is : 5
enter your choice : 4
element deleted from rear end is : 30
enter your choice : 5
queue elements :
10 20
enter your choice : 6

```

VS Stack vs Queue (Quick Compare)

Stack	Queue
LIFO (Last In First Out)	FIFO (First In First Out)
One open end (top)	Two open ends (front and rear)
push / pop	insert (enqueue) / delete (dequeue)
Example: pile of plates	Example: line for a bus or movie ticket

★ Unit 5 Quick Revision

Topic	Remember
Queue	Linear list, insert at rear , delete at front , FIFO
Real life	Movie ticket line, payment counter, printer queue, CPU time sharing
Static / Dynamic queue	Array based (fixed size) / linked list based
3 types	Simple, Circular, Double ended (Deque)
Start values	<code>front = -1, rear = -1</code>
Simple queue overflow	<code>rear = size</code>
Simple queue problem	Deleted places cannot be reused
Circular queue	Rear comes back to start; reuses free space
Circular overflow	<code>(F = 0 and R = N-1) or (F = R+1)</code>
Circular insert	If <code>R = N-1</code> then <code>R = 0</code> else <code>R = R+1</code>
Circular delete	If <code>F = N-1</code> then <code>F = 0</code> else <code>F = F+1</code>
Deque	Insert and delete at both ends; input restricted and output restricted
Circular queue uses	CPU scheduling, memory management, traffic management